

CLASP: Training-Free LLM-Assisted Source Code Watermarking via Semantic-Preserving Transformations

Rui Xu*

East China Normal University
Shanghai, China

Jiawei Chen*

East China Normal University
Shanghai, China

Weizhi Liu

East China Normal University
Shanghai, China

Zhaoxia Yin†

East China Normal University
Shanghai, China

Cong Kong

East China Normal University
Shanghai, China

Xinpeng Zhang

Fudan University
Shanghai, China

Abstract

The proliferation of open-source code and large language models (LLMs) for code generation has amplified the risks of unauthorized reuse and intellectual property infringement. Source code watermarking offers a potential solution, yet existing methods typically encode watermarks through identifiers, local code patterns, or limited handcrafted edits, leaving them vulnerable to renaming, refactoring, and adaptive watermark removal. These limitations hinder the joint achievement of robustness, capacity, generalization, and deployment efficiency. We propose **CLASP**, a Code LLM-Assisted Semantic-Preserving watermarking framework that enables training-free, plug-and-play watermarking for source code. CLASP embeds watermark bits within a fixed space of semantics-preserving transformations, enabling automated watermark insertion with higher capacity while remaining reusable across programming languages and less dependent on brittle lexical features. To recover the watermark, CLASP uses reference-code retrieval and differential comparison to identify transformation traces, avoiding task-specific model training while improving robustness to structural edits and adaptive attacks. Experiments across multiple programming languages show that CLASP consistently outperforms existing baselines in watermark extraction accuracy and robustness, while maintaining code quality under both random removal and adaptive de-watermarking attacks.

Keywords

Source Code Watermarking, Semantic-Preserving, Effectiveness

1 Introduction

With the rapid growth of the open-source ecosystem, coupled with the strong code generation capabilities of LLMs [25, 27], unauthorized use of source code has become a serious risk [17]. For instance, plagiarists may reuse open-source code with slight modifications and redistribute it under a different license, while LLMs may inadvertently reproduce copyright-protected code from their training data [28]. These risks threaten the integrity of the software ecosystem and highlight the need for reliable source code provenance. To address this problem, source code watermarking has emerged as a promising approach [31]. Its goal is to embed recoverable ownership information into code while preserving syntactic correctness, semantic equivalence, and executability [19]. The key challenge is

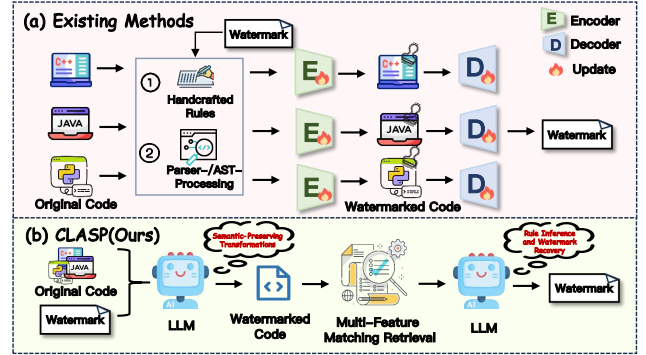


Figure 1: Comparison between existing source code watermarking methods and our CLASP, which is a training-free and plug-and-play watermarking framework for traceability.

to support reliable provenance identification without sacrificing code fidelity or practical usability. Existing methods range from handcrafted semantics-preserving code transformations supported by parser- or abstract syntax tree (AST)-based analysis and rewriting to more recent training-based frameworks [31] that improve watermark naturalness, controllability, or recovery performance.

However, existing methods still face substantial barriers to practical deployment. As summarized in Table 1, we compare them along seven dimensions following recent discussions on watermarking properties and evaluation criteria [35]. As illustrated in Figure 1, in terms of *robustness*, many methods remain vulnerable to variable renaming, code transformation, and adaptive de-watermarking attacks [19]. Regarding *efficiency*, many still rely on handcrafted rules, parser- or AST-dependent processing, additional training, or task-specific engineering, thereby limiting automation and practical deployment [12, 19, 31, 34]. With respect to *generalization*, dependence on language-specific rules or tooling hinders cross-language transfer [19, 31]. As for *capacity*, many methods support only limited watermark capacity and cannot scale to higher-capacity designs [19, 31]. Beyond these individual drawbacks, many methods also produce watermarked code with syntax errors, compilation failures, or degraded naturalness, resulting in an unsatisfactory *overall trade-off* for practical deployment.

To tackle these challenges, we propose **CLASP**, a training-free and plug-and-play framework for source code watermarking. As illustrated in Figure 1, unlike prior methods that rely on parser-/AST-dependent rewriting or trained encoder-decoder architectures,

*Both authors contributed equally to this research.

†Corresponding author.

Table 1: Qualitative comparison of code watermarking methods. Auto indicates whether the method supports automatic watermark embedding without handcrafted rules. Parser indicates whether the method avoids parser/AST-dependent processing. Language indicates whether the method can generalize across programming languages. Plug-and-Play indicates whether the method can be directly deployed without task-specific retraining or additional adaptation. Capacity is measured as the maximum number of explicitly decodable bits per function (BPF). ✓: Supported; ✗: Not supported.

Method	Robustness	Effectiveness		Generalization	Training-Free	Plug-and-Play	Trade-off	Capacity
		Auto	Parser	Language				
SrcMarker [31]	✓	✗	✗	✗	✗	✗	✗	8
CodeIP [12]	✗	✗	✓	✓	✗	✗	✗	0
RoSeMary [34]	✓	✗	✓	✓	✗	✗	✗	0
CodeMark [19]	✓	✓	✗	✗	✗	✗	✗	6
CLASP (Ours)	✓	✓	✓	✓	✓	✓	✓	32

CLASP reformulates source code watermarking into two stages: *Semantically Consistent Embedding*, which uses LLMs to perform semantics-aware watermark insertion from a fixed transformation space, and *Differential Comparison Extraction*, which recovers watermark bits through retrieval-grounded comparison against the most likely original code. This design enables automatic watermark embedding, supports cross-language deployment, improves robustness to code modifications during recovery, and avoids the need to train a dedicated decoder. Extensive experiments across multiple programming languages and attack settings show that CLASP consistently outperforms existing baselines while preserving code quality.

The main contributions of this work are as follows:

- We propose **CLASP**, a training-free and plug-and-play source code watermarking framework that combines LLM-assisted semantics-preserving embedding with retrieval-grounded extraction for code traceability and ownership verification across programming languages.
- We design *Semantically Consistent Embedding*, where prompt-driven LLMs select and apply semantics-preserving transformations from a fixed rule set, enabling automatic and generalizable watermark insertion while preserving functionality, syntax validity, and readability.
- We design *Differential Comparison Extraction*, which recovers watermark bits by retrieving the most likely original code and inferring transformation traces through code comparison, without training a dedicated decoder.
- Experiments across multiple programming languages and attack settings show that CLASP outperforms strong baselines in watermark recovery and robustness while preserving code quality and functionality.

2 Related Work

Software watermarking embeds ownership information into software artifacts for authorship or provenance verification [9]. Existing methods are typically divided into static and dynamic watermarking. Static watermarking modifies program structures, binaries, or intermediate representations [3, 5, 7, 16, 23], while dynamic watermarking encodes watermark signals into runtime behaviors or

execution states [6, 22, 29, 30]. Although effective for compiled programs, these methods are less suitable for raw source code, where syntax, semantics, and usability must be preserved.

Source code watermarking. To address this gap, recent work has shifted toward source-level watermarking. Early methods such as RopGen [20] and NatGen [4] rely on semantics-preserving transformations, while later methods introduce learned or hybrid designs. SrcMarker [31] combines rule-based transformations with neural models, ACW [18] uses manual transformations in a training-free manner, and CodeMark [19] adopts learned variable renaming with AST- and GNN-based modeling. However, existing methods still struggle to balance robustness, generalization, training-free deployment, plug-and-play usability, and high capacity.

LLM-based code transformation. Meanwhile, LLMs have shown strong ability in code generation [21, 24], transformation, and understanding, and are increasingly used for code completion, translation, and refactoring [10, 14, 15, 33]. These advances suggest that LLMs can provide a flexible interface for semantics-preserving code editing without language-specific rewriting pipelines.

Taken together, these lines of research suggest the need for a source code watermarking framework that is more flexible than traditional software watermarking and better balanced than existing source-level methods. Motivated by this, we propose CLASP, which combines LLM-assisted, semantics-preserving embedding with retrieval-grounded watermark extraction in a training-free, plug-and-play framework.

3 Threat Model

We consider a source code watermarking scenario with three parties: users, defenders, and adversaries.

Capabilities and Goals of Users. The user is a code owner who distributes function-level source code to potentially untrusted parties. The user’s primary goal is twofold: first, to ensure that the released code remains functionally correct and reusable by recipients; second, to enable post-hoc source traceability and ownership verification in the event of unauthorized reuse or redistribution.

Capabilities and Goals of Defenders. The defender is the watermarking system or its deployer, acting on behalf of the code owner to protect released source code. The defender aims to embed robust, recoverable watermark information into source code before distribution, so that any suspicious reuse or redistribution can be traced to its origin and ownership.

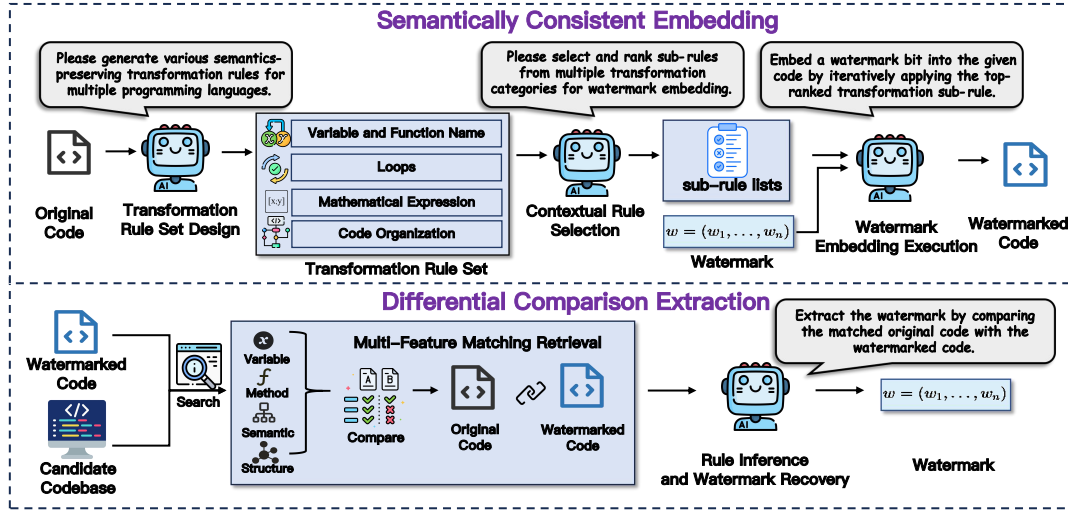


Figure 2: Overall framework of CLASP. CLASP consists of two stages: *Semantically Consistent Embedding*, which uses an LLM to select context-appropriate transformations from a fixed rule set for watermark embedding, and *Differential Comparison Extraction*, which retrieves the most likely original code candidate and recovers watermark bits through differential comparison.

In our setting, the defender starts from the original code $c_0 \in C_0$ and assigns it a watermark $w = (w_1, \dots, w_n) \in \mathcal{W}$, where n denotes the number of watermark bits and $\mathcal{W}$ denotes the set of binary watermark sequences. Using a fixed transformation rule set $\mathcal{T}$, the defender embeds w into c_0 to obtain the watermarked code $c_1 \in C_1$ before release, while preserving the functionality, syntax validity, and readability of the original code. To support post-hoc verification, the defender also maintains a candidate original codebase $\mathcal{D}$. Given a suspicious watermarked code sample after distribution, the defender retrieves the closest original candidate $\hat{c} \in \mathcal{D}$ and extracts the watermark $\hat{w}$ for verification.

Capabilities and Goals of Adversaries. The adversary is an external party who obtains a copy of the watermarked code and attempts to undermine source traceability or ownership verification. We consider a black-box adversary who does not know the specific transformation rules, the original codebase, the defender-side extraction procedure, or which code regions carry the watermark.

We consider adversaries at two capability levels. (i) *Black-box removal adversary*. The adversary applies indiscriminate functionality-preserving perturbations, such as random variable renaming or arbitrary code transformations, to disrupt potential watermark traces. (ii) *Adaptive de-watermarking adversary*. The adversary employs a general-purpose LLM to paraphrase the watermarked code, attempting to erase watermark signals through semantic-level rewriting while preserving functional correctness. In both settings, the adversary succeeds if the defender’s extraction procedure fails to recover the watermark or if ownership verification fails.

4 Method

4.1 Overview

Source code watermarking faces two key challenges: watermark embedding should be automatic and semantics-preserving across

programming languages, while watermark extraction should remain reliable under functionality-preserving code modifications without requiring a dedicated decoder. To address these challenges, we propose CLASP, which embeds watermark bits within a fixed space of semantics-preserving transformations and recovers them via differential comparison with the most likely original code reference. By constraining embedding to context-appropriate, semantics-preserving transformations rather than to unconstrained rewriting, CLASP ensures that the resulting watermarked code preserves functionality, syntactic validity, and naturalness, thereby improving watermark fidelity.

As shown in Figure 2, CLASP consists of two stages: *Semantically Consistent Embedding* and *Differential Comparison Extraction*. In the first stage, CLASP constructs a fixed transformation rule set spanning multiple categories, then uses an LLM to select and iteratively apply context-appropriate sub-rules for each watermark bit to generate watermarked code. In the second stage, given a suspicious watermarked code sample, CLASP retrieves the most likely original candidate from a defender-maintained codebase via multi-feature matching, then compares the two code samples to infer transformation traces and recover watermark bits. By coupling controlled transformation-based embedding with retrieval-grounded extraction, CLASP avoids task-specific decoder training while improving robustness under code modifications and adaptive attacks.

4.2 Semantically Consistent Embedding

Transformation Rule Set Design. CLASP first constructs a fixed set of transformation rules for watermark embedding, providing a reusable space of semantics-preserving transformations for controlled embedding and subsequent recovery. By constraining watermark insertion to this fixed transformation space rather than

Table 2: CLASP’s set of transformation rules.

Variable / Function Name		Loops	
Sub-Rule Type	Example (Original → Transformed)	Sub-Rule Type	Example (Original → Transformed)
CamelCase to snake_case	testStream() → test_stream()	for to while	for(...) → while(...) { ... }
snake_case to CamelCase	my_var → myVar	while to for	while(...) → for(...) { ... }
To PascalCase	remove() → Remove()	Flatten nested loop	for(i){...for(j){...} → for(k){...}
To UPPERCASE	value → VALUE	while to do-while	while(c){...} → do{...} while(c);
To lowercase	Value → value	Step increment	i++ → i+=1
Add suffix	data → dataVal	Reverse loop	for(i=0;...) → for(i=n-1;...)
Math Expression		Code Organization	
Sub-Rule Type	Example (Original → Transformed)	Sub-Rule Type	Example (Original → Transformed)
Group ops	x + y + z → x + (y + z)	Optimize cond.	if(x>0) return; → if(! (x>0)) return;
Mul to add	2 * x → x + x	Reorder decl.	int a; str b; → str b; int a;
Factorization	a*b + a*c → a*(b + c)	Swap params	f(a, b) → f(b, a)
Identity transform	x*x - y*y → (x - y)*(x + y)	Format spacing	int x=5; → int x = 5;
Div to reciprocal	a / b → a * (1 / b)	Add braces	if(a) return; → if(a){ return; }
Pow to mul	x*x → x * x	Reorder cond.	if(a && b) → if(b && a)
Expand distributive	a*(b + c) → a*b + a*c	Insert blank line	x=1; y=2; → x=1; \\\n y=2;
Sqrt-square	sqrt(x*x) → abs(x)	Adjust op space	x=y+z; → x = y + z;
		Inline temp var	int t=x+y; return t; → return x+y;
		Split decl.	int x=0,y=1; → int x=0; int y=1;

unconstrained rewriting, CLASP ensures that the resulting watermark code preserves functionality, syntax validity, and naturalness, thereby improving watermark fidelity. We use LLMs to construct this rule set because they support cross-language code understanding and can identify diverse semantics-preserving transformations under explicit semantic constraints, reducing language-specific bias and improving rule coverage.

Formally, we organize the transformation rule set as

$$\mathcal{T} = \bigcup_{i=1}^m \mathcal{T}_i, \quad \mathcal{T}_i = \{T_{i,1}, T_{i,2}, \dots, T_{i,k_i}\}, \quad (1)$$

where $m = 4$ denotes the number of transformation classes, $\mathcal{T}_i$ the i -th class, and k_i the number of sub-rules in that class. Each $T_{i,j}$ is a specific transformation instance for watermark embedding. The number of transformation classes determines the watermark capacity of CLASP, and higher watermark capacity can be achieved by increasing m . Rule sets for other bit configurations are provided in the Appendix.

The rule set satisfies three properties. First, all transformations are semantics-preserving. Second, the rule set is generated once, fixed for both embedding and extraction, and *kept private by the defender*, so that watermark recovery remains aligned with insertion. Third, the priority order of sub-rules within each class is predefined and fixed, providing a stable basis for subsequent rule selection and bit-wise embedding. Table 2 reports the transformation categories and representative examples for reproducibility, while *the complete deployed rule set and defender-side extraction configuration remain private*. This fixed rule set forms the basis for subsequent bit-wise watermark embedding and extraction.

Contextual Rule Selection with LLM Assistance. After constructing the fixed transformation rule set, CLASP performs rule selection for watermark embedding. Since programs differ in syntactic structure and semantic constraints, valid transformations must be selected in a context-aware manner to preserve syntax validity, functionality, and readability. Given the original code c_0 and watermark w , CLASP first determines which transformation classes are applicable to c_0 . For each applicable class $\mathcal{T}_k$, it scans the sub-rules in predefined priority order and retains the feasible ones to form the candidate list $\mathcal{L}_k(c_0) = [T_{k,1}, T_{k,2}, \dots]$. This design

combines the stability of a fixed-rule inventory with the flexibility of context-aware rule selection, thereby improving watermark embedding without language-specific engineering or additional training.

CLASP further introduces a category balancing strategy. For each watermark bit, the LLM first searches for an applicable transformation from previously unused classes to maximize category coverage; only when no new class is applicable does it reuse an existing class. This helps distribute watermark signals more evenly across the transformation space and provides a more stable basis for subsequent extraction.

Watermark Embedding Execution. After obtaining the candidate sub-rule lists $\mathcal{L}_k(c_0)$, CLASP embeds the watermark in a bit-wise manner. For each bit, the LLM selects and applies an appropriate transformation from the corresponding candidate list, so that watermark bits are encoded through controlled semantics-preserving transformations rather than unrestricted code rewriting, thereby preserving functionality, syntax validity, and readability.

Given the original code c_0 , the watermark w , and the candidate sub-rule lists $\mathcal{L}_k(c_0)$, CLASP scans the watermark sequence from left to right. For each bit position k , it selects the transformation T_k^* from $\mathcal{L}_k(c_0)$. The embedding process is defined as

$$\mathbf{c}^{(k)} = \begin{cases} T_k^*(\mathbf{c}^{(k-1)}), & \text{if } w_k = 1, \\ \mathbf{c}^{(k-1)}, & \text{if } w_k = 0, \end{cases} \quad (2)$$

$$\mathbf{c}^{(0)} = c_0, \quad \mathbf{c}_1 = \mathbf{c}^{(n)}.$$

That is, CLASP applies T_k^* only when $w_k = 1$; otherwise the code remains unchanged.

Because embedding is iterative, earlier transformations may affect later candidate rules. To improve reliability, CLASP uses a bounded retry mechanism: if the current rule is no longer applicable, it scans the remaining sub-rules in $\mathcal{L}_k(c_0)$ until a valid transformation is found or all candidates are exhausted. This improves embedding stability and supports subsequent extraction.

Table 3: Operational semantic results based on performed operations. For CSN, Pass denotes the syntax-checking rate; for MBXP/MBPP, Pass denotes the execution-passing rate. “-” indicates that the method was not evaluated on the dataset due to prohibitive computational cost. “/” indicates that the method cannot be applied to the Python language.

Metric	Method	CSN-Java	CSN-JS	MBCPP	MBJP	MBJSP	MBPP
BitAcc (%) ↑	AWT _{code} [1]	93.91	89.33	97.12	93.88	83.97	/
	CALSCode [32]	-	-	92.89	93.31	93.50	/
	SrcMarker [31]	97.26	96.34	96.04	99.44	96.94	/
	CodeMark [19]	89.14	87.07	68.75	72.54	64.30	70.30
	CLASP (Ours)	98.05	98.08	99.64	99.72	99.47	97.85
Pass (%) ↑	AWT _{code} [1]	0.18	0.51	0.00	0.00	0.00	/
	CALSCode [32]	-	-	68.19	68.65	76.77	/
	SrcMarker [31]	93.09	100.00	97.64	97.86	97.99	/
	CodeMark [19]	99.87	99.97	60.47	36.22	47.80	25.77
	CLASP (Ours)	99.85	99.11	99.35	99.31	99.87	99.69

4.3 Differential Comparison Extraction

Multi-Feature Matching Retrieval. For watermark extraction, CLASP maintains a defender-side candidate codebase $\mathcal{D}$ and retrieves the most likely original code from it before recovering watermark bits. This design differs from watermark extraction methods that rely on a trained decoder for direct recovery: by grounding extraction in retrieval and code comparison, CLASP remains training-free, is more broadly applicable across programming languages, and is more robust to code modifications and adaptive attacks, while also supporting plug-and-play deployment.

To ensure reliable retrieval, CLASP does not rely on a single matching signal. Since functionality-preserving transformations may alter lexical or structural patterns, retrieval based on only one feature can be unstable. Therefore, CLASP combines multiple complementary similarity signals, including method signature, variable usage, structural features, and normalized semantic form, to identify the most probable original code $\hat{c}$:

$$\hat{c} = \arg \max_{c_i \in \mathcal{D}} \left[\alpha \text{Sim}_m(c_i, c_1) + \beta \text{Sim}_v(c_i, c_1) + \gamma \text{Sim}_s(c_i, c_1) + \delta \text{Sim}_{\text{sem}}(c_i, c_1) \right], \quad (3)$$

$$\text{Sim}_m(c_i, c_1) = 1 - \frac{\text{LevDist}(\text{name}(c_i), \text{name}(c_1))}{\max(|\text{name}(c_i)|, |\text{name}(c_1)|)}, \quad (4)$$

$$\text{Sim}_v(c_i, c_1) = \frac{|V(c_i) \cap V(c_1)|}{|V(c_i) \cup V(c_1)|}, \quad (5)$$

$$\text{Sim}_s(c_i, c_1) = \cos(\vec{f}_s(c_i), \vec{f}_s(c_1)), \quad (6)$$

$$\text{Sim}_{\text{sem}}(c_i, c_1) = 1 - \frac{\text{LevDist}(\text{norm}(c_i), \text{norm}(c_1))}{\max(|\text{norm}(c_i)|, |\text{norm}(c_1)|)}, \quad (7)$$

where Sim_m , Sim_v , Sim_s , and Sim_{sem} denote similarity scores on method signatures, variable usage, structural features, and semantic features, respectively. Here, $\text{name}(\cdot)$ denotes the function name, $V(\cdot)$ is the set of variable identifiers, $\vec{f}_s(\cdot)$ is a vector of structural token counts, and $\text{norm}(\cdot)$ is the normalized function string after whitespace removal. In all experiments, we set $\alpha = \beta = \gamma = \delta = 0.25$, selected by grid search as detailed in the Appendix.

Once the most likely original code $\hat{c}$ is identified, CLASP proceeds to infer the applied transformations through differential comparison and then recover the embedded watermark.

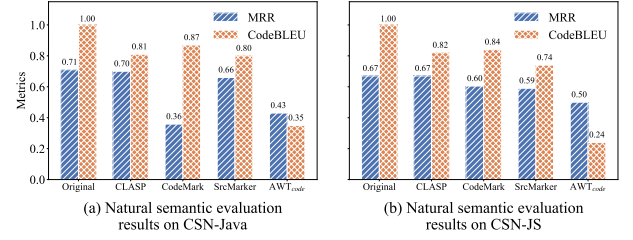


Figure 3: Natural semantics metrics for CLASP, CodeMark, SrcMarker and AWT_{code}. "Original" refers to the unwatermarked code.

Rule Inference and Watermark Recovery. After retrieving the most likely original code $\hat{c}$, CLASP uses the LLM to recover the watermark by comparing $\hat{c}$ with the watermarked code c_1 and inferring which semantics-preserving transformations have been applied. By using the LLM as the extractor, CLASP keeps the recovery process training-free, plug-and-play, and generalizable across multiple programming languages.

For each watermark bit w_k , CLASP considers the corresponding transformation class $\mathcal{T}_k$ and scans its candidate sub-rules according to the predefined priority order. Based on the syntactic and semantic consistency between $\hat{c}$ and c_1 , the LLM identifies the first sub-rule T_k^* that matches the observed code difference. In this way, each bit is recovered through contextual reasoning over transformation traces rather than direct pattern matching, thereby providing greater robustness to semantics-preserving code modifications.

The recovered bit is then defined as

$$\hat{w}_k = \begin{cases} 1, & \text{if } T_k^*(\hat{c}) \approx c_1, \\ 0, & \text{otherwise,} \end{cases} \quad (8)$$

where $\approx$ indicates that applying the candidate sub-rule T_k^* to the retrieved original code $\hat{c}$ produces code consistent with the watermarked code c_1 while preserving syntax validity and functionality.

If the current candidate sub-rule does not yield a valid match, CLASP continues scanning the remaining candidate sub-rules within the same transformation class until a consistent transformation is found or a retry limit is reached. This mechanism further improves the stability of watermark recovery. The final recovered watermark is $\hat{w} = [\hat{w}_1, \dots, \hat{w}_n]$.

5 Evaluation

In this section, we evaluate CLASP. We first describe the experimental setup in Section 5.1. For CLASP, we evaluate its watermark accuracy and fidelity (Section 5.2), robustness (Section 5.3), effectiveness (Section 5.4), capacity (Section 5.5), and ablation study (Section 5.6).

5.1 Experiment Setup

Datasets and Preprocessing. To evaluate CLASP across languages, we use CodeSearchNet and MBXP benchmarks covering C, C++,

Table 4: Performance under black-box removal and adaptive de-watermarking attacks. BBR denotes black-box removal attacks, and ADW denotes adaptive de-watermarking. Here, T denotes random code transformation, V denotes random variable substitution, and Pass is the percentage of attacked code that passes the test cases. “/” indicates that the method cannot be applied to the Python language.

Dataset	Metric	Method	No Atk.	BBR							ADW	
				T@1	T@2	T@3	V@25%	V@50%	V@75%	V@100%	GPT-4o	
MBJP	BitAcc (%) ↑	SrcMarker [31]	99.44	86.37	82.19	78.56	81.47	77.88	67.79	56.62	50.12	
		CodeMark [19]	72.54	68.88	67.31	64.28	67.99	62.11	57.90	49.32	48.75	
		CLASP (Ours)	99.72	88.84	86.84	84.46	96.03	95.75	90.61	82.29	76.93	
	Pass (%) ↑	SrcMarker [31]	97.86	75.42	75.30	74.94	74.58	73.40	72.33	71.26	98.46	
		CodeMark [19]	36.22	36.22	36.10	36.22	35.99	35.99	36.10	35.51	12.59	
		CLASP (Ours)	99.31	93.92	86.74	82.32	91.57	88.81	87.97	87.29	99.17	
	CodeBLEU (%) ↑	SrcMarker [31]	-	53.51	52.36	51.89	54.22	53.79	53.44	52.93	48.80	
		CodeMark [19]	-	75.62	66.90	62.06	83.57	78.35	73.12	67.14	61.57	
		CLASP (Ours)	-	85.00	76.74	70.00	84.18	77.82	68.88	73.45	47.85	
MBJSP	BitAcc (%) ↑	SrcMarker [31]	96.94	76.47	70.64	69.70	81.93	79.33	75.63	67.06	50.72	
		CodeMark [19]	64.30	59.47	54.27	54.77	60.79	58.34	54.74	49.28	49.94	
		CLASP (Ours)	99.47	93.10	88.08	88.05	95.68	93.62	90.90	87.86	79.02	
	Pass (%) ↑	SrcMarker [31]	97.99	96.86	97.24	96.61	96.24	96.36	95.23	94.35	99.25	
		CodeMark [19]	47.80	46.80	46.68	46.68	46.93	46.68	46.68	46.55	0.50	
		CLASP (Ours)	99.87	98.37	98.12	96.99	96.49	93.60	90.97	89.34	99.87	
	CodeBLEU (%) ↑	SrcMarker [31]	-	49.24	48.72	48.54	49.31	49.10	48.91	48.43	49.80	
		CodeMark [19]	-	66.12	58.80	55.08	76.09	72.07	68.26	62.80	54.47	
		CLASP (Ours)	-	75.03	65.90	60.06	72.85	64.47	60.27	57.02	48.55	
MBCPP	BitAcc (%) ↑	SrcMarker [31]	96.04	85.21	77.65	73.56	83.05	79.52	72.64	60.18	49.18	
		CodeMark [19]	68.75	64.23	61.85	60.11	63.94	58.84	57.04	49.51	49.15	
		CLASP (Ours)	99.64	92.64	91.36	89.20	94.93	92.67	91.66	90.35	83.48	
	Pass (%) ↑	SrcMarker [31]	97.64	79.84	79.45	79.71	78.66	77.62	74.48	73.95	99.74	
		CodeMark [19]	60.47	47.25	46.73	46.99	46.34	46.86	46.60	45.42	2.09	
		CLASP (Ours)	99.35	95.16	92.15	91.23	90.18	88.48	88.09	86.91	99.74	
	CodeBLEU (%) ↑	SrcMarker [31]	-	44.88	44.86	44.99	44.08	43.81	43.57	42.98	50.75	
		CodeMark [19]	-	69.60	59.05	52.72	78.55	72.64	66.94	59.67	52.04	
		CLASP (Ours)	-	66.65	59.05	54.49	71.21	64.64	61.48	58.25	4.88	
MBPP	BitAcc (%) ↑	SrcMarker [31]	/	/	/	/	/	/	/	/	/	
		CodeMark [19]	70.30	50.62	50.46	50.44	49.46	49.02	49.72	49.92	50.18	
		CLASP (Ours)	97.85	93.85	92.96	90.80	89.13	95.82	94.16	92.82	79.72	
	Pass (%) ↑	SrcMarker [31]	/	/	/	/	/	/	/	/	/	
		CodeMark [19]	25.77	25.70	25.66	25.54	25.36	25.67	25.46	25.26	7.80	
		CLASP (Ours)	99.69	95.64	95.43	95.22	95.11	95.85	91.70	88.28	99.38	
	CodeBLEU (%) ↑	SrcMarker [31]	/	/	/	/	/	/	/	/	/	
		CodeMark [19]	-	60.45	57.08	52.30	60.45	57.08	52.30	47.04	0.45	
		CLASP (Ours)	-	64.01	61.10	57.14	52.10	65.22	68.28	67.44	41.19	

Java, JavaScript, and Python. For semantic evaluation, we use CSN-Java and CSN-JS [13], in which functions are paired with natural-language descriptions from open-source projects. For execution-based validation, we use MBXP [2], including MBCPP, MBJP, MBJSP, and MBPP. Each function embeds a 4-bit watermark. Detailed settings and results on GitHub subsets are provided in Appendix. **Baselines and LLMs.** We choose CodeMark, SrcMarker, AWT_{code}, and CALS_{code} proposed in SrcMarker as baselines. AWT_{code} and CALS_{code} are adapted from AWT [1] and CALS [32], both originally developed for natural language watermarking. AWT_{code} keeps the AWT architecture but is trained on source code datasets, while CALS_{code} replaces BERT [8] with CodeBERT [11] to better fit source code data. For CLASP, we use GPT-4o for embedding and extraction;

results with DeepSeek-V3 and Gemini 1.5 Pro are reported in the Appendix.

5.2 Watermark Accuracy and Fidelity

Metrics. We evaluate watermarking performance from two perspectives: **accuracy** and **fidelity**. For watermark accuracy, we use Bit Accuracy (BitAcc), the percentage of watermark bits correctly extracted. For watermark fidelity, we evaluate whether the watermarked code preserves its operational correctness after transformation. Specifically, for the CSN datasets, we use syntax checking, since the functions are not independently compilable; following prior work, we adopt tree-sitter to detect AST errors. For the MBXP datasets, we use the pass rate, defined as the fraction of watermarked programs that pass all unit tests. In addition, following prior

Table 5: Comparison of runtime and per-sample economic cost across different watermarking methods.

Method	AWT _{code} [1]	SrcMarker [31]	CodeMark [19]	CLASP (Ours)
BitAcc (%) ↑	93.91	97.26	89.14	98.05
Pass (%) ↑	0.18	93.09	99.87	99.85
Training Time (h) ↓	61.50	13.32	13.87	0
Embedding Time (s) ↓	0.1055	0.0741	0.0387	3.3333
Extraction Time (s) ↓	0.0023	0.0034	0.0074	1.3333
Total Time (h) ↓	61.50	13.32	13.87	5.85
Economic Cost (\$) ↓	0.0123	0.0027	0.0029	0.0020

studies, we assess natural semantic fidelity using CodeBLEU [26] and MRR. CodeBLEU measures code similarity using syntax, data flow, and n-gram matching, while MRR evaluates whether watermarking affects the rank of code retrieval for natural language queries, with a higher score indicating better semantic retention. For MRR computation, we use a fine-tuned CodeBERT [11].

Accuracy results. Table 3 shows that CLASP achieves the best overall watermark extraction accuracy across the evaluated datasets. Compared with CodeMark, SrcMarker, AWT_{code}, and CALS_{code}, CLASP consistently obtains the highest BitAcc, demonstrating stronger watermark recovery effectiveness across programming languages. On the two CSN datasets, CLASP outperforms all baselines by clear margins. On the MBXP benchmarks, its advantage is even more pronounced, where it consistently achieves near-perfect BitAcc while several baselines show noticeable degradation.

Operational semantic results. The operational semantic results are shown in Table 3. AWT_{code} performs poorly in execution-based evaluation, with nearly zero pass rates on MBXP datasets, indicating that it often fails to preserve executable functionality after watermark embedding. CALS_{code} improves over AWT_{code}, but still fails a substantial portion of test cases. SrcMarker achieves relatively strong operational fidelity, but still suffers occasional syntax or execution failures. Although CodeMark maintains high pass rates on the CSN datasets, its execution pass rates drop sharply on MBXP benchmarks, suggesting that it struggles to preserve actual runtime behavior. In contrast, CLASP consistently achieves near-perfect pass rates across datasets, demonstrating that its semantics-preserving embedding strategy maintains code validity and functionality while enabling accurate watermark recovery.

Natural semantic results. Figure 3 shows the results on natural semantic fidelity. Higher MRR and CodeBLEU indicate better preservation of naturalness and semantic consistency after watermark embedding. Overall, CLASP remains closest to the original code and achieves the best balance between MRR and CodeBLEU on both datasets, showing stronger natural semantic fidelity than the baselines. In contrast, SrcMarker and AWT_{code} lead to more noticeable degradation in naturalness, while CodeMark presents a different trade-off: although it preserves relatively high CodeBLEU, its MRR drops more substantially, suggesting that it is less effective at maintaining retrieval-based natural semantic consistency.

5.3 Robustness

Random removal attack. To evaluate the **robustness** of CLASP, we consider a black-box removal adversary that knows code transformations may be used for watermark embedding, but does not know the exact transformation rules or extraction procedure. The

attacker performs random functionality-preserving edits by renaming 25%, 50%, 75%, and 100% of variables, and applying up to 1, 2, or 3 random code transformations to each program. As shown in Table 4, the BitAcc of all methods decreases as the attack intensity increases. Nevertheless, CLASP consistently maintains higher BitAcc than the baselines across all MBXP datasets under both transformation-based and identifier-level attacks, while also preserving high pass rates after attack. This advantage is especially clear under variable-substitution attacks, where CLASP remains much more stable than SrcMarker and CodeMark as the renaming ratio increases. Because CLASP introduces a category balancing strategy during embedding, it does not rely on a single transformation pattern to carry watermark signals. As a result, even if random transformation attacks destroy watermark evidence in some code regions, the remaining transformation categories can still support accurate recovery. This makes CLASP more robust to functionality-preserving random edits while maintaining code executability.

Adaptive de-watermarking. We further evaluate CLASP under a stronger adaptive adversary that is aware of the general LLM-based embedding setting, but does not know the exact watermarking strategy. In this setting, the attacker uses GPT-4o to rewrite the watermarked code while preserving its intended functionality, aiming to remove watermark signals through higher-level paraphrasing and reorganization. As shown in Table 4, CLASP consistently achieves the highest BitAcc across MBXP datasets under this stronger attack setting, while also maintaining high pass rates after rewriting. By contrast, the baselines show much sharper degradation, with several BitAcc results approaching random-guessing level. This is because adaptive rewriting changes not only identifiers and local syntax, but also higher-level expression patterns and code organization, making methods that rely more heavily on surface-form signals easier to weaken. By contrast, CLASP performs extraction through differential comparison with the most likely original code candidate, allowing it to recover watermark bits from persistent transformation-level differences rather than exact lexical matches, thereby maintaining high watermark recoverability and code executability under adaptive de-watermarking attacks.

5.4 Effectiveness

5.4.1 Economic Cost. Metrics. We compare CLASP with the baselines using per-sample cost, including total time (training, embedding, and extraction time) and economic cost. In addition, to highlight the trade-off between efficiency and watermark quality, we also report Bit Accuracy (BitAcc) and Pass rate, where Pass denotes the syntax-checking rate on the CSN-Java dataset. SrcMarker, AWT_{code}, and CodeMark require non-trivial computational resources for model training, so we estimate their training costs using a common market rental price of \$2/hour for GPU usage. Due to the extremely high runtime of CALS_{code} (over 342 hours), we do not include it in the comparison.

Results. Table 5 shows a clear trade-off between inference efficiency and watermark quality. Although CLASP requires longer embedding and extraction time than the baselines at inference time, it achieves the highest BitAcc and near-perfect syntax pass rate, demonstrating substantially better plug-and-play watermarking quality. In contrast, methods such as AWT_{code}, SrcMarker, and

Table 6: Plug-and-play evaluation of CLASP under cross-LLM embedding and extraction, measured by Bit Accuracy (BitAcc) across MBXP datasets.

Watermarker	Extractor	MBJP (%)	MBJSP (%)	MBCPP (%)	MBPP (%)
GPT-4o	GPT-4o	99.72	99.47	99.64	97.85
	DeepSeek-V3	99.30	99.72	98.85	95.51
	Gemini-1.5-pro	99.26	99.72	99.48	95.38
DeepSeek-V3	DeepSeek-V3	98.69	99.56	98.75	98.37
	GPT-4o	95.59	97.62	98.28	96.73
	Gemini-1.5-pro	99.62	95.67	98.28	98.99
Gemini-1.5-pro	Gemini-1.5-pro	98.70	98.19	98.13	97.37
	GPT-4o	98.79	98.43	98.54	96.38
	DeepSeek-V3	99.34	99.75	98.98	96.96

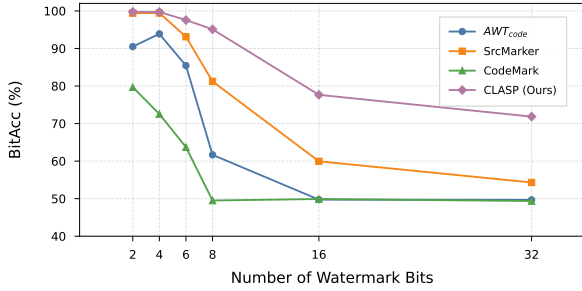


Figure 4: BitAcc changes with respect to watermark capacity (number of watermark bits embedded into a single function).

CodeMark are faster once trained, but their performance is either less accurate or less stable in terms of syntax correctness. At the same time, because CLASP avoids any upfront training costs, it still achieves the lowest overall cost and the shortest end-to-end runtime among the methods compared. These results show that CLASP offers a favorable practical trade-off: it sacrifices some inference speed to obtain much stronger watermark extraction accuracy and syntax preservation, while remaining more economical overall than training-based baselines.

5.4.2 Generalization. Metrics. To evaluate the plug-and-play capability of CLASP, we conduct cross-LLM experiments in which watermark embedding and extraction are performed by different LLMs. Specifically, we instantiate the watermarker and extractor with three representative LLMs, namely **GPT-4o**, **DeepSeek-V3**, and **Gemini 1.5 Pro**, and evaluate all cross-model combinations. We use Bit Accuracy (BitAcc) as the metric, which measures the percentage of watermark bits correctly recovered. A method with strong plug-and-play capability should maintain high extraction accuracy even when the embedding and extraction stages are carried out by different LLMs.

Results. Table 6 shows that CLASP maintains consistently high BitAcc across different watermarker–extractor combinations, indicating that it does not rely on a fixed model pairing and remains stable across heterogeneous LLM settings. This is because both embedding and extraction are grounded in the same fixed transformation space, rather than tied to a model-specific decoder or parameterization. Moreover, as shown in Table 3, several baseline methods exhibit limited language generalization due to their

reliance on language-specific parsing and transformation mechanisms, and thus cannot be applied to Python or C programs, making them non-plug-and-play. In contrast, CLASP supports multiple programming languages while maintaining strong watermark recovery and execution fidelity. These results demonstrate strong cross-language generalization and plug-and-play capability.

5.5 Capacity

We conduct capacity experiments on the MBJP dataset and compare the BitAcc of different methods across watermark capacities of 2, 4, 6, 8, 16, and 32 bits. As shown in Figure 4, the BitAcc of all methods decreases as the number of embedded bits increases, indicating that reliable watermark embedding and extraction become more difficult at higher capacities. Notably, after 8 bits, the other three methods all drop to near-random performance, suggesting that their watermark embedding and extraction fail under high-capacity settings. In contrast, CLASP maintains high BitAcc not only at low-bit settings but also under 16-bit and 32-bit watermarking. These results show that CLASP supports higher watermark capacity while preserving reliable recovery performance.

5.6 Ablation Study

Metrics. We conduct ablation experiments on MBJP to evaluate the necessity of the retrieval, embedding, and extraction components in CLASP. For retrieval, we compare the full multi-feature retrieval of CLASP with several single-feature variants, including name-only, variable-only, structure-only, and semantic-only retrieval, as well as the original-code setting. For embedding, we remove the fixed transformation rule set. For extraction, we replace retrieval-grounded differential comparison with direct bit prediction without using the original code. We report Top-1 and Top-5 for retrieval variants, and BitAcc for final watermark recovery.

Results. Table 7 reports the ablation results. For retrieval, the full multi-feature retrieval achieves the same BitAcc as using the original code, whereas all single-feature variants perform worse, indicating that combining complementary features is necessary to reliably identify the original reference and support downstream extraction. For embedding, removing the fixed transformation rule set results in a sharp drop in BitAcc, indicating that the shared transformation space is necessary to align watermark insertion with subsequent recovery. For extraction, replacing retrieval-grounded differential comparison with direct bit prediction without using the original code yields near-random performance, indicating that accurate watermark recovery relies on *Differential Comparison Extraction*. Overall, these results confirm the necessity of retrieval, fixed-rule embedding, and differential comparison in CLASP.

6 Conclusion

We present CLASP, an LLM-assisted framework for source code watermarking that reformulates watermark embedding and extraction as semantic reasoning and retrieval-grounded verification. Unlike prior methods that rely on handcrafted rules, parser-/AST-dependent rewriting, or retraining, CLASP enables automatic watermarking without model fine-tuning or language-specific engineering. Across extensive experiments, CLASP demonstrates strong robustness, effectiveness, generalization, training-free deployment,

Table 7: Ablation results on MBJP. “Multi-feature retrieval” denotes the full retrieval setting of CLASP, and “–” indicates that retrieval accuracy is not applicable.

Component	Variant	Top-1 (%) ↑	Top-5 (%) ↑	BitAcc (%) ↑
Retrieval	Original code	100.00	100.00	99.72
	Multi-feature retrieval (full CLASP)	100.00	100.00	99.72
	Name-only retrieval	54.00	96.40	77.10
	Variable-only retrieval	61.60	80.80	81.25
	Structure-only retrieval	23.00	46.00	64.25
	Semantic-only retrieval	0.80	2.80	50.30
Embedding	w/o fixed transformation rule set	–	–	31.33
Extraction	Direct bit prediction without original code	–	–	52.01

plug-and-play usability, and high capacity, yielding a favorable overall trade-off among these desirable properties. These results show that CLASP provides a practical and broadly applicable solution for source code watermarking.

References

- [1] Sahar Abdelnabi and Mario Fritz. 2021. Adversarial Watermarking Transformer: Towards Tracing Text Provenance with Data Hiding. In *2021 IEEE Symposium on Security and Privacy*. 121–140.
- [2] Ben Athiwaratkun, Sanjay Krishna Gouda, Zijian Wang, Xiaopeng Li, Yuchen Tian, Ming Tan, Wasi Uddin Ahmad, Shiqi Wang, Qing Sun, and Mingyue Shang. 2022. Multi-lingual Evaluation of Code Generation Models. In *The Eleventh International Conference on Learning Representations*.
- [3] Vivek Balachandran, Ng Wee Keong, and Sabu Emmanuel. 2014. Function Level Control Flow Obfuscation for Software Security. In *Proceedings of the Eighth International Conference on Complex, Intelligent and Software Intensive Systems*. 133–140.
- [4] Saikat Chakraborty, Toufique Ahmed, Yangruibo Ding, Premkumar T. Devanbu, and Baishakhi Ray. 2022. NatGen: Generative Pre-training by “Naturalizing” Source Code. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 18–30.
- [5] Jianping Chen, Kui Li, Wanzhi Wen, Weixu Chen, and Chenxue Yan. 2018. Software Watermarking for Java Program Based on Method Name Encoding. In *Proceedings of the International Conference on Advanced Intelligent Systems and Informatics 2017*. 865–874.
- [6] Zhe Chen, Chunfu Jia, and Donghui Xu. 2017. Hidden Path: Dynamic Software Watermarking Based on Control Flow Obfuscation. In *2017 IEEE International Conference on Computational Science and Engineering and IEEE International Conference on Embedded and Ubiquitous Computing*. 443–450.
- [7] Christian Collberg and Tapas Ranjan Sahoo. 2005. Software Watermarking in the Frequency Domain: Implementation, Analysis, and Attacks. *Journal of Computer Security* 13, 5 (2005), 721–755.
- [8] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*. 4171–4186.
- [9] Ayan Dey, Sukriti Bhattacharya, and Nabendu Chaki. 2019. Software Watermarking: Progress and Challenges. *INAE Letters* 4, 1 (2019), 65–75.
- [10] Bosheng Ding, Chengwei Qin, Ruochen Zhao, Tianze Luo, Xinze Li, Guizhen Chen, Wenhan Xia, Junjie Hu, Luu Anh Tuan, and Shafiq Joty. 2024. Data Augmentation Using LLMs: Data Perspectives, Learning Paradigms and Challenges. In *Findings of the Association for Computational Linguistics: ACL 2024*. 1679–1705.
- [11] Zhangyin Feng, Daya Guo, Duyu Tang, Nan Duan, Xiaocheng Feng, Ming Gong, Linjun Shou, Bing Qin, Ting Liu, and Daxin Jiang. 2020. CodeBERT: A Pre-trained Model for Programming and Natural Languages. In *Findings of the Association for Computational Linguistics: EMNLP 2020*. 1536–1547.
- [12] Batu Guan, Yao Wan, Zhangqian Bi, Zheng Wang, Hongyu Zhang, Pan Zhou, and Lichao Sun. 2024. CodeLP: A Grammar-Guided Multi-Bit Watermark for Large Language Models of Code. In *Findings of the Association for Computational Linguistics: EMNLP 2024*. 9243–9258.
- [13] Hamel Husain, Ho-Hsiang Wu, Tiferet Gazit, Miltiadis Allamanis, and Marc Brockschmidt. 2019. *CodeSearchNet Challenge: Evaluating the State of Semantic Code Search*. arXiv:1909.09436 doi:10.48550/arXiv.1909.09436
- [14] Prithwish Jana, Piyush Jha, Haoyang Ju, Gautham Kishore, Aryan Mahajan, and Vijay Ganesh. 2024. Cotran: An LLM-based Code Translator Using Reinforcement Learning with Feedback from Compiler and Symbolic Execution. In *ECAI 2024*. 4011–4018.
- [15] Juyong Jiang, Fan Wang, Jiasi Shen, Sungju Kim, and Sunghun Kim. 2024. *A Survey on Large Language Models for Code Generation*. arXiv:2406.00515 doi:10.48550/arXiv.2406.00515
- [16] Honggoo Kang, Yonghwi Kwon, Sangjin Lee, and Hyungjoon Koo. 2021. SoftMark: Software Watermarking via a Binary Function Relocation. In *Annual Computer Security Applications Conference*. 169–181.
- [17] Raphael Khoury, Anderson R. Avila, Jacob Brunelle, and Baba Mamadou Camara. 2023. *How Secure is Code Generated by ChatGPT?* arXiv:2304.09655 doi:10.48550/arXiv.2304.09655
- [18] Boquan Li, Mengdi Zhang, Peixin Zhang, Jun Sun, Xingmei Wang, and Zirui Fu. 2024. *ACW: Enhancing Traceability of AI-Generated Codes Based on Watermarking*. arXiv:2402.07518 doi:10.48550/arXiv.2402.07518
- [19] Wei Li, Borui Yang, Yujie Sun, Suyu Chen, Yuting Chen, and Liyao Xiang. 2025. CodeMark: Contextual and Natural Watermarking for Tracing Code Snippet Provenance. *IEEE Transactions on Dependable and Secure Computing* (2025), 7077–7090.
- [20] Zhen Li, Guenevere Chen, Chen Chen, Yayi Zou, and Shouhuai Xu. 2022. Ropgen: Towards Robust Code Authorship Attribution via Automatic Coding Style Transformation. In *Proceedings of the 44th International Conference on Software Engineering*. 1906–1918.
- [21] Aixin Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, et al. 2024. *Deepseek-v3 Technical Report*. arXiv:2412.19437 doi:10.48550/arXiv.2412.19437
- [22] Haoyu Ma, Chunfu Jia, Shijia Li, Wantong Zheng, and Dinghao Wu. 2019. Xmark: Dynamic Software Watermarking Using Collatz Conjecture. *IEEE Transactions on Information Forensics and Security* 14, 11 (2019), 2859–2874.
- [23] A. Monden, H. Iida, K. Matsumoto, K. Inoue, and K. Torii. 2000. A Practical Method for Watermarking Java Programs. In *Proceedings of the 24th Annual International Computer Software and Applications Conference*. 191–197.
- [24] OpenAI. 2022. *ChatGPT: Optimizing Language Models for Dialogue*. Retrieved 2026-04-01 from https://openai.com/blog/chatgpt
- [25] Kexin Pei, David Bieber, Kensen Shi, Charles Sutton, and Pengcheng Yin. 2023. Can Large Language Models Reason about Program Invariants?. In *Proceedings of the International Conference on Machine Learning*. 27496–27520.
- [26] Shuo Ren, Daya Guo, Shuai Lu, Long Zhou, Shujie Liu, Duyu Tang, Neel Sundaresan, Ming Zhou, Ambrosio Blanco, and Shuai Ma. 2020. *CodeBLEU: A Method for Automatic Evaluation of Code Synthesis*. arXiv:2009.10297 doi:10.48550/arXiv.2009.10297
- [27] Disha Shrivastava, Hugo Larochelle, and Daniel Tarlow. 2023. Repository-Level Prompt Generation for Large Language Models of Code. In *Proceedings of the International Conference on Machine Learning*. 31693–31715.
- [28] Zhensu Sun, Xiaoning Du, Fu Song, Mingze Ni, and Li Li. 2022. Coprotector: Protect Open-source Code Against Unauthorized Training Usage with Data Poisoning. In *Proceedings of the ACM Web Conference 2022*. 652–660.
- [29] Zhenzhou Tian, Qinghua Zheng, Ting Liu, Ming Fan, Eryue Zhuang, and Zijiang Yang. 2015. Software Plagiarism Detection with Birthmarks Based on Dynamic Key Instruction Sequences. *IEEE Transactions on Software Engineering* 41, 12 (2015), 1217–1235.
- [30] Yilong Wang, Daofu Gong, Bin Lu, Fei Xiang, and Fenlin Liu. 2018. Exception Handling-Based Dynamic Software Watermarking. *IEEE Access* 6 (2018), 8882–8889.
- [31] Borui Yang, Wei Li, Liyao Xiang, and Bo Li. 2024. SrcMarker: Dual-Channel Source Code Watermarking via Scalable Code Transformations. In *Proceedings of the 2024 IEEE Symposium on Security and Privacy*. 4088–4106.
- [32] Xi Yang, Jie Zhang, Kejiang Chen, Weiming Zhang, Zehua Ma, Feng Wang, and Nenghai Yu. 2022. Tracing Text Provenance via Context-Aware Lexical Substitution. In *Proceedings of the AAAI Conference on Artificial Intelligence*. 11613–11621.
- [33] Mingxuan Zhang, Bo Yuan, Hanzhe Li, and Kangming Xu. 2024. LLM-Cloud Complete: Leveraging Cloud Computing for Efficient Large Language Model-based Code Completion. *Journal of Artificial Intelligence General Science* 5, 1 (2024), 295–326.
- [34] Ruisi Zhang, Neusha Javidnia, Nojan Sheybani, and Farinaz Koushanfar. 2025. *Robust and secure code watermarking for large language models via ml/crypto codesign*. arXiv:2502.02068 doi:10.48550/arXiv.2502.02068
- [35] Xuandong Zhao, Sam Gunn, Miranda Christ, Jaiden Fairoze, Andres Fabrega, Nicholas Carlini, Sanjam Garg, Sanghyun Hong, Milad Nasr, Florian Tramer, Somesh Jha, Lei Li, Yu-Xiang Wang, and Dawn Song. 2025. SoK: Watermarking for AI-Generated Content. In *Proceedings of the 2025 IEEE Symposium on Security and Privacy*. 2621–2639.